MBLP131 · Hafta 7 · Ders Notları

İşletim Sistemleri Temelleri

Sanal Bellek ve Dosya Sistemleri

4 GB'lık bir program 2 GB RAM'de nasıl çalışır ve dosyalar gerçekte nerede durur?

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Bu haftanın sonunda yapabilmeniz gerekenler

1. Sanal bellek fikrini ve bir programın yalnızca bir kısmı RAM'deyken neden çalışabildiğini açıklamak.
2. İsteğe bağlı sayfalamayı, geçerli–geçersiz bitini ve işletim sisteminin bir sayfa hatasını adım adım nasıl işlediğini anlatmak.
3. Sayfa hatalarıyla etkin erişim süresini ve belirli bir yavaşlamayı aşmayan en büyük hata oranını hesaplamak.
4. Yazınca kopyala yöntemini ve `fork()` ile bağlantısını açıklamak.
5. FIFO, OPT ve LRU sayfa değiştirme için çerçeve tablolarını çizmek, sayfa hatalarını saymak; Belady anomalisini ve saat algoritmasını açıklamak.
6. Boşa çalışmayı (thrashing) ve çalışma kümesi modelini açıklamak.
7. Dosya özniteliklerini, dosya işlemlerini, açık dosya tablolarını ve sıralı, doğrudan ve indeksli erişimi (blok hesabıyla birlikte) anlatmak.
8. Tek düzeyli, iki düzeyli, ağaç ve döngüsüz çizge dizinlerini karşılaştırmak; mutlak ve göreli yolları kullanmak; düğümleri (inode), sabit ve sembolik bağlantıları açıklamak.

İçindekiler

1 6. Haftadan 7. Haftaya 1

2 Sanal Bellek 2

3 İsteğe Bağlı Sayfalama 2

4 Sayfa Değiştirme 7

5 Boşa Çalışma ve Çalışma Kümesi 11

6 Dosya Kavramı 12

7 Erişim Yöntemleri 14

8 Dizin Yapıları 16

9 Linux Ne Gösterir: Düğümler ve Bağlantılar 18

10 Özet 20

11 Sözlük 20

12 Kendiniz Deneyin 21

13 Tekrar Soruları 21

14 Vize Tekrarı: 1–7. Haftalar 22

15 İleriye Bakış: 8. Hafta ve Sonrası 23

1 6. Haftadan 7. Haftaya

Geçen hafta **MMU** mantıksal adresleri fiziksel adreslere çevirdi, **sayfalama** (paging) belleği sayfalara ve çerçevelere böldü, bir **sayfa tablosu** eşlemeyi kaydetti ve **TLB** son çevirileri önbellekte tuttu. Bunların hepsi örtük olarak *çalışan bir sürecin her sayfasının RAM'de olduğunu* varsayıyordu. Bu hafta bu varsayımı bırakıyoruz: bir sayfa diskte de durabilir ve ancak gerektiğinde belleğe getirilebilir. **Sanal bellek** (virtual memory) adı verilen bu fikir, bir bilgisayarın RAM'inin alabileceğinden daha çok program çalıştırmasını sağlar. İkinci yarıda işletim sisteminin diğer büyük depolama soyutlamasına geçiyoruz: **dosya**.

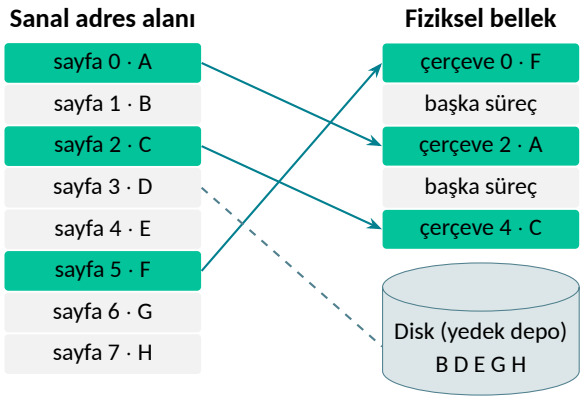
2 Sanal Bellek

2.1 Fikir

Gerçek bir programa bakarsanız büyük kısmının nadiren kullanıldığını görürsünüz: sıra dışı hataları işleyen kod, pek az kişinin tıkladığı menü seçenekleri, $10\,000$ girdilik yer ayrılıp yalnızca 50'si kullanılan diziler. Kullanılan kodun bile tamamı aynı anda gerekmez. Programlar **başvuru yerelliği** (locality of reference) gösterir: herhangi bir kısa süre içinde program sayfalarının yalnızca birkaçıyla çalışır (o anki fonksiyon, onun yerel değişkenleri, dizinin işlenmekte olan kısmı).

Sanal bellek

Sanal bellek, programın gördüğü belleği (*sanal* ya da mantıksal adres alanını) makinenin fiziksel belleğinden ayırır. RAM'de yalnızca o an gereken sayfalar tutulur; geri kalanlar **yedek depo** (backing store) ya da takas alanı (swap space) denen bir disk bölgesinde kalır. Bu yüzden sanal adres alanı fiziksel bellekten çok daha *büyük* olabilir.



Şekil 1: RAM'de yalnızca A, C ve F sayfaları var; programın diğer sayfaları diskte bekliyor.

Kazançlar azımsanacak gibi değildir:

- Bir program artık fiziksel belleğin boyutuyla sınırlı değildir. (64 bitlik Linux'ta her sürecin 128 TiB'lık bir sanal adres alanı vardır.)
- Her süreç daha az RAM kaplar, bu yüzden belleğe aynı anda daha çok süreç sığar: *çoklu programlama derecesi* ve onunla birlikte işlemci kullanımı artar (1. ve 4. haftalar).
- Bir programı yüklemek ya da takaslamak için daha az G/Ç gerekir; programlar daha hızlı başlar.
- Sayfalar süreçler arasında paylaşılabilir (paylaşılan kütüphaneler ve `fork()` sonrasında yazınca kopyala, Kısım 3.5).

3 İsteğe Bağlı Sayfalama

3.1 Tembel yükleme

İşletim sistemi hangi sayfaları RAM'e koyacağına nasıl karar verir? En basit strateji, program başlamadan önce tamamını yüklemektir. Her modern genel amaçlı işletim sisteminin kullandığı seçenek ise **isteğe bağlı sayfalamadır** (demand paging): bir sayfa ancak ona ilk kez başvurulduğunda yüklenir. Bu, *tembel yükleme* (lazy loading) denen genel bir programlama fikrinin bir örneğidir: bir işi gerekmeden önce asla yapma.

Programın tamamını hevesle yüklemeye kıyasla isteğe bağlı sayfalama daha az G/Ç ve daha az bellek gerektirir, programları daha hızlı başlatır ve RAM'e daha çok süreç sığdırır. Bedeli, her sayfanın ilk kullanımının bir *sayfa hatasına* yol açmasıdır.

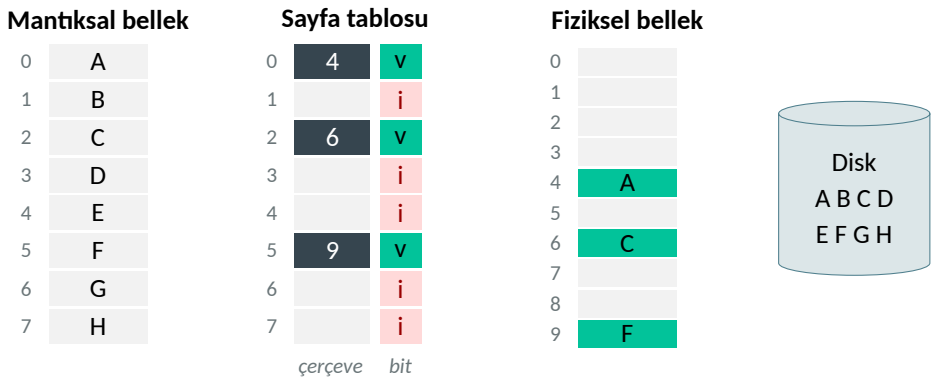
İşletim sisteminin tek tek sayfaları disk ile bellek arasında taşıyan kısmına **sayfalayıcı** (pager) denir. (Daha eski *takasçı*, swapper, sözcüğü *süreçlerin tamamını* taşımayı anlatır.) *Saf* isteğe bağlı sayfalamada bir süreç bellekte hiç sayfası olmadan başlar; daha ilk komutu bir hataya yol açar.

3.2 Geçerli--geçersiz biti

Donanımın bir sayfanın bellekte olup olmadığını anlamasının bir yolu olmalıdır. Her sayfa tablosu girdisinde bir **geçerli–geçersiz biti** (valid–invalid bit) bulunur:

- **v (valid, geçerli):** sayfa sürece aittir *ve* bir çerçevededir; girdi çerçeve numarasını tutar.
- **i (invalid, geçersiz):** sayfa ya şu an bellekte değildir ya da sürecin adres alanının parçası bile değildir.

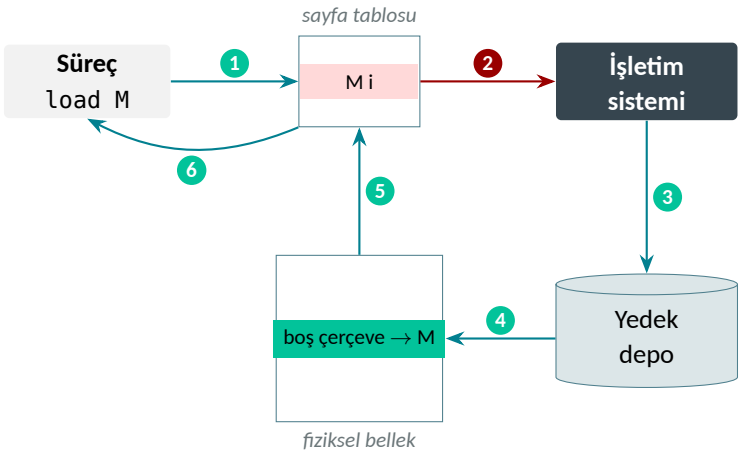
Başlangıçta her girdi i yapılır. Bir süreç yalnızca geçerli sayfalara eriştiği sürece 6. haftadaki gibi, tam hızda çalışır.



Şekil 2: A, C ve F sayfaları 4, 6 ve 9 numaralı çerçevelerde (bit v); diğer tüm girdiler geçersiz. Her sayfanın diskte bir kopyası var.

3.3 Bir sayfa hatasının işlenmesi

Süreç, biti i olan bir sayfaya dokunursa ne olur? MMU adresi çeviremez ve bir **sayfa hatası** (page fault) üretir: işletim sistemine bir tuzak (trap, 2. hafta). İşletim sistemi bundan sonra şu adımları izler.



Şekil 3: Sayfa hatası işlemenin altı adımı.

1. **Başvuru.** Süreç, M sayfasına başvuran bir komut yürütür. Sayfa tablosu girdisi i der.
2. **Tuzak.** MMU bir sayfa hatası üretir. İşletim sistemi sürecin durumunu kaydeder ve bir iç tabloya bakar: adres yasal mı? Değilse süreç sonlandırılır (Linux'ta: *segmentation fault*). Yasal ama bellekte değilse devam edilir.

3. **Bulma.** İşletim sistemi M sayfasının yedek depodaki yerini ve boş bir çerçeve bulur (boş çerçeve yoksa bir sayfayı *değiştirmesi* gerekir, Kısım 4).
4. **Okuma.** Sayfanın çerçeveye okunması için bir disk işlemi başlatır. Bu milisaniyeler sürer; bu arada işlemci zamanlayıcısı (4. hafta) başka bir süreci çalıştırır, hata veren süreç ise *bekliyor* durumundadır (3. hafta).
5. **Güncelleme.** Disk kesmesi geldiğinde işletim sistemi çerçeve numarasını sayfa tablosu girdisine yazar ve biti v yapar.
6. **Yeniden başlatma.** Süreç yeniden hazır duruma getirilir ve hataya yol açan komut *baştan yeniden başlatılır*. Bu kez erişim başarılı olur; süreç bir şey olduğunu fark edemez (yavaşlama dışında).

Küçük ve büyük hatalar. Her sayfa hatası disk gerektirmez. Sayfa yepyeni bir bellekse (örneğin `malloc` ile ayrılmış bir diziye ilk dokunuş), işletim sisteminin yalnızca boş bir çerçeve bulup sıfırlarla doldurması yeter. Linux buna **küçük hata** (minor fault) der. **Büyük hata** (major fault) ise disk G/Ç'si gerektirir; örneğin bir program dosyasının bir sayfasını ya da diske takaslanmış bir sayfayı okumak için. Büyük hatalar binlerce kat daha pahalıdır.

3.4 Sayfa hatalarıyla etkin erişim süresi

p, bir bellek erişiminin sayfa hatasına yol açma olasılığı, yani **sayfa hatası oranı** (page-fault rate) olsun ($0 \le p \le 1$). Bellek erişim süresi t_m ve sayfa hatası hizmet süresi t_{fault} ile **etkin erişim süresi** (effective access time, EAT) şu ağırlıklı ortalamadır:

$$\text{EAT} = (1 - p)\, t_m + p\, t_{\text{fault}}.$$

Bu, 6. haftadaki TLB formülüyle tam olarak aynı yapıdadır: olasılığıyla ağırlıklandırılmış hızlı durum artı olasılığıyla ağırlıklandırılmış yavaş durum. Hata hizmet süresi tuzağı, durumun kaydedilmesini, disk okumasını (açık ara en büyük kısım) ve sürecin yeniden başlatılmasını içerir.

Çözümlü örnek: sayfa hatalarıyla EAT

$t_m = 200$ ns ve $t_{\text{fault}} = 8$ ms $= 8\,000\,000$ ns alalım ve binde bir erişimin hata verdiğini varsayalım, $p = 0.001$:

$$\begin{aligned}\text{EAT} &= 0.999 \times 200 \text{ ns} + 0.001 \times 8\,000\,000 \text{ ns} \\ &= 199.8 \text{ ns} + 8\,000 \text{ ns} = 8\,199.8 \text{ ns} \approx 8.2\ \mu\text{s}.\end{aligned}$$

Bellek $8199.8/200 \approx 41$ kat yavaşlamıştır!

Yavaşlamanın %10'un altında kalması için p ne kadar küçük olmalı? EAT $< 1.1\, t_m = 220$ ns olmalıdır:

$$200 + p\,(8\,000\,000 - 200) < 220 \quad \Longrightarrow \quad p < \frac{20}{7\,999\,800} \approx 2.5 \times 10^{-6}.$$

Bu, en fazla $1/p \approx 400\,000$ bellek erişiminde bir sayfa hatası demektir. (Sayılar `code/page_replacement.py` ile denetlendi.)

Ders şudur: sayfa hatası oranı son derece küçük olmalıdır ve uygulamada küçük *olmasının* tek nedeni yerelliktir. Bir SSD ile (t_{fault} yaklaşık 0.1 ms mertebesinde) ceza daha küçüktür, ama aynı akıl yürütme geçerlidir.

3.5 Yazınca kopyala

3. haftada, ebeveyninin kopyası olarak bir çocuk süreç oluşturan `fork()` çağrısını gördük. Ebeveynin her sayfasını kopyalamak yavaş ve çoğu zaman anlamsız olurdu: pek çok çocuk hemen `exec()` çağırır ve tüm belleğini yeni bir programla değiştirir. Sanal bellek daha iyi bir çözüm sunar: **yazınca kopyala** (copy-on-write, COW).



Şekil 4: Yazınca kopyala: sayfalar bir süreç yazana kadar paylaşılır; yalnızca yazılan sayfa kopyalanır.

`fork()` sonrasında ebeveyn ve çocuk tüm sayfalarını paylaşır ve işletim sistemi bu sayfaları salt okunur olarak işaretler. Okumak bedavadır. Süreçlerden biri paylaşılan bir sayfaya *yazdığında* donanım bir koruma hatası üretir; işletim sistemi yalnızca o sayfayı kopyalar, yazan sürece kendi (artık yazılabilir) kopyasını verir ve yazma işlemini yeniden başlatır. Hiç yazılmayan sayfalar hiç kopyalanmaz. Mekanizma, isteğe bağlı sayfalamada kullandığımız tuzak-ve-yeniden-başlat düzeneğinin aynısıdır.

3.6 Uygulama: sayfa hatalarını izlemek

Bu kısımdaki programlar, bu notların yanındaki `code/` klasöründedir. Hepsini `make` ile derleyin (`gcc -Wall -O2` kullanır).

Örnek 1: isteğe bağlı sayfalama. Program `malloc()` ile 64 MB ayırır, sonra her sayfaya iki kez birer bayt yazar. Çekirdek her süreç için sayfa hatalarını sayar; `getrusage()` bu sayacı okur. Listede programın özü var; sonuçları yazdıran tam sürüm `page_faults.c` dosyasıdır.

```c
static long minor_faults(void)          /* çekirdeğin sayacı */
{
    struct rusage ru;
    getrusage(RUSAGE_SELF, &ru);
    return ru.ru_minflt;
}

int main(void)
{
    long page = sysconf(_SC_PAGESIZE);  /* 4096 bayt    */
    long size = 64 * MB;                /* 16384 sayfa  */

    long f0 = minor_faults();
    char *a = malloc(size);             /* yalnızca sanal */
    long f1 = minor_faults();
    for (long i = 0; i < size; i + = page)
        a[i] = 1;                       /* ilk dokunuş  */
    long f2 = minor_faults();
```

```
    for (long i = 0; i < size; i + = page)
        a[i] + = 1;                      /* artık bellekte */
    long f3 = minor_faults();
    /* f1-f0, f2-f1, f3-f2 yazdırılır */
}
/* programın tamamı: code/page_faults.c */
```

Test makinemizdeki çıktı (Linux, 4 KB'lık sayfalar):

```
$ ./page_faults
page size 4096 bytes; array 64 MB = 16384 pages
after malloc       :      1 new faults
first touch (write):  16383 new faults
second touch       :      0 new faults
```

Çıktıyı okumak:

- `malloc()` kendisi yalnızca 1 hataya yol açar: *sanal* alan ayırır ve (ilk sayfaya) yalnızca küçük bir kayıt başlığı yazar.
- İlk geçiş dokunduğu her sayfa için bir kez hata verir: 16.383 hata, artı `malloc`'un zaten dokunduğu 1 sayfa, 16.384 sayfanın tamamını verir.
- İkinci geçiş 0 hataya yol açar: her sayfa artık bellektedir.

Bunların hepsi *küçük* hatalardır: çekirdeğin yalnızca sıfırlanmış çerçeveler vermesi gerekti. `/usr/bin/time -v` aracı tüm çalışmanın sayaçlarını raporlar (programın ve C kütüphanesinin yüklenmesindeki hatalar dahil):

```
$ /usr/bin/time -v ./page_faults
...
   Maximum resident set size (kbytes): 66964
   Major (requiring I/O) page faults: 0
   Minor (reclaiming a frame) page faults: 16463
```

Saydam büyük sayfaların (transparent huge pages) tüm bellek için etkin olduğu sistemlerde çekirdek bunun yerine 2 MB'lık sayfalar eşleyebilir; o zaman çok daha az hata görürsünüz. (`/usr/bin/time` yoksa aracı `sudo apt install time` ile kurun.)

Örnek 2: yazınca kopyala. Ebeveyn 32 MB'lık bir tamponu doldurur ve `fork()` yapar. Çocuk önce her sayfayı okur, sonra her sayfaya yazar. Listede programın özü var; Örnek 1'deki `minor_faults()` yardımcı fonksiyonunu da içeren tam sürüm `code/cow_fork.c` dosyasıdır.

```
char *buf = malloc(SIZE);              /* 32 MB = 8192 sayfa */
memset(buf, 'A', SIZE);                /* sayfalar ebeveynin */

pid_t pid = fork();
if (pid == 0) {                        /* ---- çocuk ----    */
    long f0 = minor_faults();
    for (long i = 0; i < SIZE; i + = page)
        sum + = buf[i];                 /* OKU: paylaşılan    */
    long f1 = minor_faults();
    for (long i = 0; i < SIZE; i + = page)
```

```
        buf[i] = 'B';                 /* YAZ: sayfa kopyası */
    long f2 = minor_faults();
    /* f1 - f0 ve f2 - f1 yazdırılır */
    exit(0);
}
wait(NULL);
printf("parent still sees '%c'\n", buf[0]);
/* programın tamamı: code/cow_fork.c */
```

```
$ ./cow_fork
buffer: 8192 pages of 4096 bytes
child read  all pages:      0 faults
child write all pages:   8192 faults (copy-on-write)
parent still sees  'A'  (its copy was never changed)
```

Paylaşılan sayfaları okumanın bedeli yoktur; yazmanın bedeli ise sayfa başına tam olarak bir hatadır, toplam 8192, çünkü her sayfa ilk yazıldığında kopyalanır. Ebeveyn hâlâ özgün verisini görür. (`fork()` öncesindeki `fflush(stdout)` önemlidir: o olmasaydı ebeveynin çıktı tamponunda bekleyen metin çocuğa da kopyalanır ve iki kez basılırdı.)

4 Sayfa Değiştirme

4.1 Bellek dolduğunda

İşletim sistemi belleği bilerek *fazla dağıtır*: tüm süreçlerin sanal belleklerinin toplamı RAM'den büyüktür; tüm sayfaların aynı anda gerekmeyeceğine bahse girilir. Er ya da geç bir sayfa hatası olur ve boş çerçeve yoktur. İşletim sistemi bu durumda çıkarılacak bir **kurban** (victim) sayfa seçmelidir. Sayfa hatası yordamı şu hâle gelir:

1. İstenen sayfanın diskteki yerini bul.
2. Boş bir çerçeve bul. Yoksa bir **sayfa değiştirme algoritması** (page-replacement algorithm) ile bir kurban çerçeve seç; kurban değiştirilmişse diske yaz; onun sayfa tablosu girdisini geçersiz işaretle.
3. İstenen sayfayı boşalan çerçeveye oku; sayfa tablosunu güncelle.
4. Süreci yeniden başlat.

Değişiklik (kirli) biti. Dikkat edilmezse bir değiştirme *iki* disk aktarımına mal olur: biri kurbanı dışarı yazmak, biri yeni sayfayı içeri okumak için. Bu yüzden donanım her sayfa için bir **değişiklik biti** (modify bit, dirty bit) tutar; sayfaya her yazıldığında bu bit kendiliğinden kurulur. Program kodunun bir sayfası gibi *temiz* bir kurbanın diskte aynısı zaten vardır ve doğrudan atılabilir.

Algoritmaları nasıl karşılaştırırız? Her algoritmayı aynı **başvuru dizisi** (reference string: bir programın başvurduğu sayfa numaraları dizisi) ve aynı çerçeve sayısıyla çalıştırır ve sayfa hatalarını sayarız. Bu kısım boyunca klasik

$$7,\ 0,\ 1,\ 2,\ 0,\ 3,\ 0,\ 4,\ 2,\ 3,\ 0,\ 3,\ 2,\ 1,\ 2,\ 0,\ 1,\ 7,\ 0,\ 1$$

dizisini, başlangıçta hepsi boş 3 çerçeveyle kullanıyoruz. Aşağıdaki tablolarda nane rengi yeni yüklenen sayfayı, soluk sütunlar sayfa hatalarını (F = fault), gri sütunlar isabetleri gösterir. Tüm tablolar `code/` klasöründeki Python benzetimcisiyle (`page_replacement.py`) üretildi.

4.2 FIFO: ilk giren ilk çıkar

Kurban, bellekte en uzun süredir duran sayfadır. FIFO, sayfa numaralarından oluşan bir kuyrukla kolayca gerçeklenir.

başvuru	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
çerçeve 0	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	0	0	0	0	**7**	7	7
çerçeve 1		**0**	0	0	0	**3**	3	3	**2**	2	2	2	2	**1**	1	1	1	1	**0**	0
çerçeve 2			**1**	1	1	1	**0**	0	0	**3**	3	3	3	3	**2**	2	2	2	2	**1**
hata?	F	F	F	F		F	F	F	F	F	F			F	F			F	F	F

Şekil 5: 3 çerçeveyle FIFO: **15 sayfa hatası**.

İlk üç hata **zorunlu** (compulsory, soğuk başlangıç) hatalardır: hiçbir algoritma 7, 0 ve 1'i ilk kez yüklemekten kaçınamaz. Sonra 2, 7'nin (en eski) yerine geçer, 0 bir isabettir ve 3, 0'ın yerine geçer, çünkü artık en eski sayfa 0'dır; oysa 0'a hemen bir sonraki başvuruda yine gerek vardır. FIFO bir sayfanın ne zaman geldiğini bilir, hâlâ kullanılıp kullanılmadığını bilmez.

4.3 OPT: en iyi algoritma

Kurban, gelecekte *en uzun süre kullanılmayacak* olan sayfadır. Bu algoritma (OPT ya da MIN, Bélády 1966), herhangi bir başvuru dizisi ve çerçeve sayısı için olası en düşük sayfa hatası sayısını verir.

başvuru	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
çerçeve 0	**7**	7	7	**2**	2	2	2	2	2	2	2	2	2	2	2	2	2	**7**	7	7
çerçeve 1		**0**	0	0	0	0	0	**4**	4	4	**0**	0	0	0	0	0	0	0	0	0
çerçeve 2			**1**	1	1	**3**	3	3	3	3	3	3	3	**1**	1	1	1	1	1	1
hata?	F	F	F	F		F		F			F			F				F		

Şekil 6: 3 çerçeveyle OPT: **9 sayfa hatası**, olası en az sayı.

Örneğin 4 numaralı sayfaya başvuruda (8. konum) çerçevelerde 2, 0 ve 3 vardır. İleriye bakınca 2'nin bir sonraki kullanımı 9. konumda, 3'ünkü 10. konumda, 0'ınki 11. konumdadır; bu yüzden 0 değiştirilir. Sorun açıktır: işletim sistemi gelecekteki başvuru dizisini bilemez. OPT bir *ölçüt* (yardstick) olarak kullanılır: "algoritmamız bu iş yükünde en iyinin %10 yakınında".

4.4 LRU: en uzun süredir kullanılmayan

LRU (least recently used) yakın geçmişi yakın geleceğin tahmini olarak kullanır: kurban, *en uzun süredir kullanılmamış* sayfadır. Bir başka deyişle, zamanda geriye bakan OPT'dir.

başvuru	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
çerçeve 0	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	**1**	1	1	1	1	1	1
çerçeve 1		**0**	0	0	0	0	0	0	0	**3**	3	3	3	3	3	**0**	0	0	0	0
çerçeve 2			**1**	1	1	**3**	3	3	**2**	2	2	2	2	2	2	2	2	**7**	7	7
hata?	F	F	F	F		F		F	F	F	F			F		F		F		

Şekil 7: 3 çerçeveyle LRU: **12 sayfa hatası**.

4 numaralı sayfaya başvuruda çerçevelerde yine 2, 0 ve 3 vardır, ama bu kez geriye bakarız: en uzun süre önce kullanılan 2'dir, bu yüzden 2 değiştirilir (OPT 0'ı değiştirmişti). LRU yerellik sayesinde iyi çalışır.

Tam LRU, *her* bellek erişiminde donanım desteği gerektirir: ya sayfa tablosu girdisine yazılan bir zaman damgası ya da başvurulan sayfanın en üste taşındığı bir sayfa numarası yığıtı (stack). İkisi de gerçek donanım için çok pahalıdır; bu yüzden sistemler LRU'ya *yaklaşır* (Kısım 4.7).

4.5 Karşılaştırma

Algoritma	Kurban şu sayfadır...	Hata (3 çerçeve)	Gerçeklenebilir mi?	Belady anomalisi?
FIFO	en uzun süre önce yüklenen	15	evet, çok ucuz	olabilir
OPT	gelecekte en geç kullanılacak	9	hayır (geleceği ister)	asla
LRU	en uzun süredir kullanılmayan	12	yaklaşık olarak	asla

Sınav ipucu: sütun sütun çalışın ve her başvurudan sonra çerçevelerin içeriğini yazın. OPT için dizide *ileriye*, LRU için *geriye* bakın; FIFO için yalnızca geliş sırasını hatırlayın.

4.6 Belady anomalisi

Sezgi, bir sürece daha çok çerçeve vermenin onun sayfa hatası sayısını asla artıramayacağını söyler. FIFO için sezgi yanılır. $1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5$ başvuru dizisini düşünün:

başvuru	1	2	3	4	1	2	5	1	2	3	4	5
çerçeve 0	**1**	1	1	**4**	4	4	**5**	5	5	5	5	5
çerçeve 1		**2**	2	2	**1**	1	1	1	1	**3**	3	3
çerçeve 2			**3**	3	3	**2**	2	2	2	2	**4**	4
hata?	F	F	F	F	F	F	F			F	F	

başvuru	1	2	3	4	1	2	5	1	2	3	4	5
çerçeve 0	**1**	1	1	1	1	1	**5**	5	5	5	**4**	4
çerçeve 1		**2**	2	2	2	2	2	**1**	1	1	1	**5**
çerçeve 2			**3**	3	3	3	3	3	**2**	2	2	2
çerçeve 3				**4**	4	4	4	4	4	**3**	3	3
hata?	F	F	F	F			F	F	F	F	F	F

Şekil 8: Aynı dizide FIFO: **3 çerçeveyle 9 hata** (üstte), ama **4 çerçeveyle 10 hata** (altta).

Bu, **Belady anomalisidir** (Belady's anomaly). LRU ve OPT bunu asla göstermez. Bunlar **yığıt algoritmaları** (stack algorithms) sınıfındandır: n çerçeveyle bellekte bulunan sayfalar kümesi, $n+1$ çerçeveyle bulunanların

her zaman bir alt kümesidir (LRU için: en son kullanılan n sayfa her zaman en son kullanılan $n+1$ sayfanın içindedir). Bu yüzden bir çerçeve eklemek hiçbir zaman hata ekleyemez. Aynı dizide LRU 3 çerçeveyle 10, 4 çerçeveyle 8 hata verir.

László Bélády

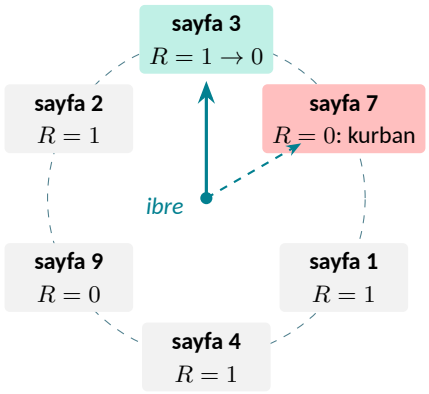
Macaristan doğumlu bilgisayar bilimci László Bélády IBM Research'te çalıştı. 1966'da en iyi değiştirme algoritmasını (MIN) yayımladı; 1969'da da R. A. Nelson ve G. S. Shedler ile birlikte, bugün onun adını taşıyan anomaliyi anlatan makaleyi yazdı.

4.7 LRU'ya yaklaşmak: başvuru biti ve saat

Birçok işlemci her sayfa için bir **başvuru biti** (reference bit) sağlar: sayfa her okunduğunda ya da yazıldığında donanım bu biti 1 yapar. İşletim sistemi biti okuyabilir ve 0'a çekebilir. Bit, bir sayfanın *ne zaman* kullanıldığını söylemez, ama *bit son kez sıfırlandığından beri* kullanılıp kullanılmadığını söyler.

İkinci şans (second-chance) ya da **saat** (clock) algoritması çerçeveleri bir çember üzerine dizer; bir işaretçi, "saat ibresi", sıradaki adayı gösterir:

- İbrenin altındaki sayfanın $R = 1$ ise sayfaya ikinci bir şans verilir: $R = 0$ yapılır ve ibre ilerletilir.
- $R = 0$ ise sayfa, ibre son geçtiğinden beri kullanılmamıştır: kurban odur.



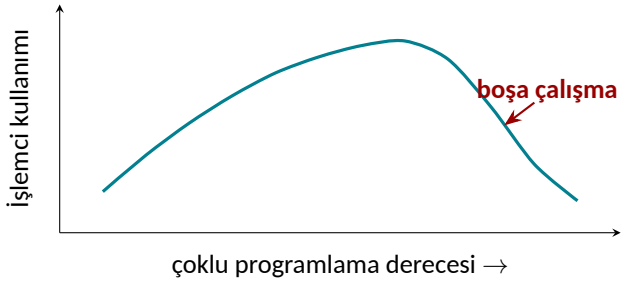
Şekil 9: İbre 3. sayfada ($R = 1$): biti sıfırlanır ve ibre 7. sayfaya ($R = 0$) ilerler; 7. sayfa değiştirilir.

Bütün bitler 1 ise ibre bitleri sıfırlayarak çemberde bir tur atar ve algoritma FIFO'ya dönüşür. Gerçek sistemler bu fikrin geliştirilmiş biçimlerini kullanır; örneğin Linux "etkin" (active) ve "etkin olmayan" (inactive) sayfa listeleri tutar.

5 Boşa Çalışma ve Çalışma Kümesi

5.1 Boşa çalışma

Bir sürecin etkin olarak kullandığı sayfa sayısının çerçeve sayısından fazla olduğunu düşünün. Süreç hata verir, çıkardığı sayfa yakında yine gereken bir sayfadır, bu yüzden yine hata verir, yine ve yine. Zamanının çoğunu yürütmeye değil sayfalamaya harcayan bir süreç **boşa çalışıyordur** (thrashing).



Şekil 10: Bellekteki süreç sayısına karşı işlemci kullanımı (kavramsal eğri, ölçülmüş veri değildir).

Eğri tehlikeyi açıklar. Süreç eklemek önce işlemci kullanımını artırır (bazıları G/Ç beklerken çalıştırılacak başka süreçler vardır). Belli bir noktadan sonra süreçlerin her birine yetecek kadar çerçeve kalmaz; sayfa hataları patlar, süreçler sayfalama diski için kuyruğa girer ve işlemci kullanımı çöker. Boşta duran işlemciye bakıp daha da çok süreç kabul eden saf bir zamanlayıcı durumu kötüleştirir: bir kısır döngü. Çare, çoklu programlama derecesini *düşürmek* (bazı süreçleri askıya almak ya da diske takaslamak) ya da fiziksel bellek eklemektir. RAM'i az, tarayıcı sekmesi çok, disk ışığı sürekli yanan ve faresi zar zor hareket eden bir bilgisayar kullandıysanız boşa çalışmayı görmüşsünüzdür.

5.2 Çalışma kümesi modeli

Bir sürecin t anındaki **çalışma kümesi** (working set) $\mathrm{WS}(t, \Delta)$, en son Δ başvurusunda kullanılan farklı sayfaların kümesidir; Δ'ya çalışma kümesi penceresi denir. Çalışma kümesi, programın o anki yerelliğine yaklaşır.



Şekil 11: Çalışma kümesi programın yerelliğini izler: t_1 anında 5 sayfa, t_2 anında yalnızca 2 sayfa.

$\mathrm{WSS}_i = |\mathrm{WS}_i|$, i. sürecin çalışma kümesi boyutuysa, toplam çerçeve talebi $D = \sum_i \mathrm{WSS}_i$ olur. D kullanılabilir çerçeve sayısı m'yi aşarsa boşa çalışma başlar ve işletim sistemi, çerçeveler boşalana kadar bir süreci askıya almalıdır. Δ'nın seçimi önemlidir: çok küçükse yerelliğin bir kısmını kaçırır; çok büyükse birkaç yerelliği birden kapsar ve çerçeve israf eder.

Uygulamada daha basit bir akrabası *sayfa hatası sıklığı* (page-fault frequency) denetimidir: işletim sistemi her sürecin hata oranını ölçer, oran çok yüksekse sürece daha çok çerçeve verir, çok düşükse çerçevelerinden bazılarını geri alır.

6 Dosya Kavramı

Ana bellek uçucudur ve her şeyi tutamayacak kadar küçüktür. Bu yüzden işletim sistemi elektrik kesildikten sonra da varlığını sürdüren ikinci bir depolama soyutlaması sunar: **dosya** (file), ikincil bellekte kaydedilmiş, adı olan bir ilişkili bilgiler topluluğu. Kullanıcı açısından dosya, kaydedilebilecek en küçük depolama birimidir: diske veriyi ancak bir dosyanın içinde yazabilirsiniz.

6.1 Dosya öznitelikleri

Her dosyanın içeriğinin yanı sıra **öznitelikleri** (attributes, üst veri) vardır. Sağdaki sütun, Kısım 9.1'teki `stat` çıktısından alınmış gerçek bir dosyanın değerlerini gösterir.

Öznitelik	Anlamı	Örnek (`notes.txt`)
Ad	İnsanın okuyabileceği ad; Linux'ta diğer özniteliklerle birlikte değil, dizinde saklanır.	`notes.txt`
Tanımlayıcı	Dosyanın dosya sistemi içindeki benzersiz numarası.	düğüm (inode) 368662
Tür	Normal dosya, dizin, sembolik bağlantı, aygıt, …	normal dosya
Konum	Dosyanın veri bloklarının aygıttaki yeri.	düğümdeki blok işaretçileri
Boyut	Bayt cinsinden güncel boyut (ve ayrılmış bloklar).	6 bayt (512 B'lık 8 blok)
Koruma	Kim okuyabilir, yazabilir, çalıştırabilir.	`-rw-rw-r--`, sahibi `student`
Zaman damgaları	Oluşturma, son erişim, son değişiklik, son durum değişikliği.	Access / Modify / Change / Birth

İşletim sistemi bu öznitelikleri her dosya için tutulan ve **dosya denetim bloğu** (file control block, FCB) denen bir yapıda saklar. UNIX ve Linux'ta dosya denetim bloğu **düğümdür** (inode, Kısım 9).

6.2 Dosya işlemleri

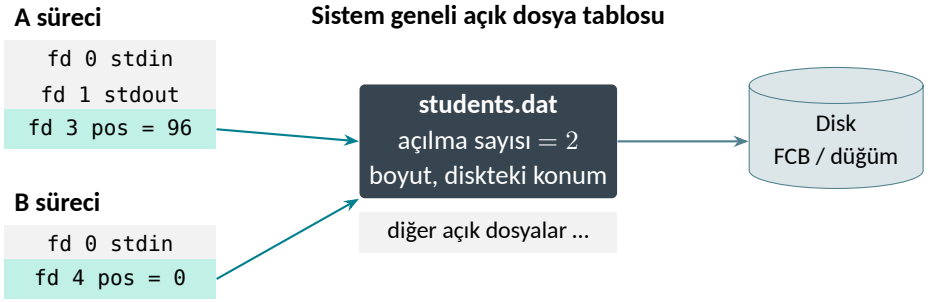
Dosya bir *soyut veri türüdür*: ona uygulayabileceğiniz işlemlerle tanımlanır. Temel işlemler ve karşılık gelen Linux sistem çağrıları şunlardır:

İşlem	İşletim sisteminin yaptığı	Linux çağrısı
create (oluştur)	Dosya sisteminde yer bulur ve dizine bir girdi ekler.	`open(…, O_CREAT)`
open (aç)	Dizini bir kez arar, izinleri denetler, bir *dosya tanımlayıcısı* döndürür.	`open()`
read (oku)	Dosyadan, geçerli konumdan başlayarak baytları kopyalar.	`read()`
write (yaz)	Dosyaya, geçerli konumdan başlayarak baytları kopyalar.	`write()`
seek (konumlan)	Geçerli konumu taşır (yeniden konumlandırma); veri aktarılmaz.	`lseek()`
close (kapat)	Açık dosya tablosu girdisini bırakır.	`close()`
delete (sil)	Dizin girdisini kaldırır; artık kullanılmayınca yeri boşaltır.	`unlink()`
truncate (kırp)	İçeriği siler ama öznitelikleri korur.	`truncate()`

Diğer işlemler (sonuna ekleme, yeniden adlandırma, kopyalama) bunlardan kurulabilir. Silme işleminin adına dikkat edin, `unlink()`: dosyanın bir *adını* kaldırır; Kısım 8.3 bunun neden her zaman veriyi silmekle aynı şey olmadığını açıklar.

6.3 Açık dosya tabloları

Her bir `read()` için dizini aramak ve izinleri denetlemek yavaş olurdu. Bunun yerine `open()` bu işi bir kez yapar ve sonucu bellekteki tablolarda saklar; sürecin sonraki tüm çağrılarda kullandığı küçük bir tamsayı, **dosya tanımlayıcısı** (file descriptor) döndürür. 0, 1 ve 2 numaralı tanımlayıcılar bir program başladığında zaten açıktır: standart girdi, standart çıktı ve standart hata (2. hafta).



Şekil 12: Süreç başına tablolar her sürecin dosya işaretçisini tutar; sistem geneli tablo paylaşılan bilgileri ve bir açılma sayısını tutar.

- **Süreç başına açık dosya tablosunda**, o sürecin açık her dosyası için bir girdi bulunur; girdide **dosya işaretçisi** (file pointer, geçerli konum) ve erişim kipi (okuma, yazma, ...) vardır.
- **Sistem geneli açık dosya tablosunda**, açık her dosya için, onu açmış tüm süreçlerin paylaştığı bir girdi bulunur; girdide süreçten bağımsız bilgiler vardır: diskteki konum, boyut ve bir **açılma sayısı** (open count). `close()` sayıyı bir azaltır; sayı 0'a inince girdi kaldırılır.

Bu, ders kitabındaki modeldir. Linux bir düzey daha ekler (tanımlayıcı $\rightarrow$ konumu tutan "açık dosya tanımı", open file description $\rightarrow$ bellekteki düğüm), ama fikir aynıdır.

6.4 Dosya türleri ve uzantılar

İşletim sistemi ve uygulamalar bir dosyanın ne tür veri içerdiğini bilmelidir. En yaygın teknik, türü adın içine bir **uzantı** (extension) olarak koymaktır:

Dosya türü	Olağan uzantılar	İçerik
Kaynak kod	`.c .py .java`	bir programlama dilinde program metni
Nesne / çalıştırılabilir	`.o .exe` (Linux'ta yok)	makine kodu
Betik	`.sh .bat`	bir komut yorumlayıcısı için komutlar
Metin / belge	`.txt .pdf .docx`	metin, biçimli belgeler
Çoklu ortam	`.jpg .mp3 .mp4`	görüntü, ses, video
Arşiv	`.zip .tar.gz`	çoğunlukla sıkıştırılmış birden çok dosya

Windows bir dosyayı hangi programın açacağına uzantıya göre karar verir. UNIX ve Linux'ta uzantı yalnızca insanlar ve uygulamalar için bir *ipucudur*; çekirdek onunla ilgilenmez. `file` komutu adı görmezden gelir ve içeriğe, özellikle ilk birkaç bayta bakar; pek çok biçimde bu baytlar sabit bir **sihirli sayıdır** (magic number): bir ELF çalıştırılabilir dosyası `7F 45 4C 46` baytlarıyla, bir JPEG görüntüsü `FF D8 FF` ile başlar.

```
$ file file_access.c
file_access.c: C source, ASCII text
$ file file_access
file_access: ELF 64-bit LSB pie executable, x86-64, version 1...
$ file photo.txt
photo.txt: JPEG image data, JFIF standard 1.01, aspect ratio,...
$ file students.dat
students.dat: data
```

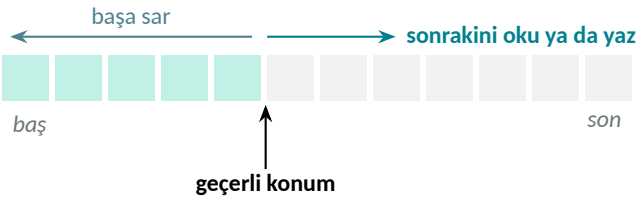
Burada `photo.txt` aslında yanıltıcı adlı bir JPEG resmidir; `students.dat` ise Örnek 4'ün yazdığı ikili kayıt dosyasıdır (uzun satırlar `...` ile kesildi).

7 Erişim Yöntemleri

Bir dosyadaki bilgi nasıl okunabilir? İşletim sistemi birkaç standart **erişim yöntemi** (access method) sunar.

7.1 Sıralı erişim

Bilgi sırayla, bir kayıt ötekinin ardından işlenir. *Sonrakini oku* (read next) dosyanın sonraki kısmını döndürür ve dosya işaretçisini ilerletir; *sonrakini yaz* (write next) sona ekler ve ilerletir; *başa sar* (rewind) başa döner. Bu, manyetik teybin modelidir; çoğu programın da modelidir: düzenleyiciler, derleyiciler ve medya oynatıcılar girdilerini baştan sona okur.



Şekil 13: Sıralı erişim.

7.2 Doğrudan (göreli) erişim

Dosya, sabit boyutlu *blok* ya da *kayıtlardan* oluşan numaralı bir dizi olarak görülür ve herhangi biri herhangi bir sırada okunup yazılabilir: n*'yi oku*, n*'ye yaz* ya da n*'ye git* ve ardından sonrakini oku. Bu, herhangi bir bloğa yaklaşık aynı sürede ulaşabilen bir diske uygun düşer. Veritabanları ve rezervasyon sistemleri doğrudan erişime ihtiyaç duyar: bir uçuştaki 17C koltuğunu bulmak için ondan önceki bütün koltukları okumak istemezsiniz.

Blok numarası n, dosyanın başına *görelidir*: blok 0, blok 1 ve böyle devam eder. İşletim sistemi göreli blok numaralarını fiziksel disk bloklarına çevirir; kullanıcının dosyanın aygıtta nerede durduğunu bilmesi gerekmez. Doğrudan erişimli bir dosyada sıralı erişim, bir geçerli konum cp tutularak kolayca taklit edilir: *sonrakini oku*, "cp bloğunu oku, sonra $cp \leftarrow cp + 1$" demektir.

Çözümlü örnek: 1000. kayıt nerede?

Bir dosya, 0'dan numaralanmış, her biri $R = 100$ bayt olan öğrenci kayıtları tutuyor. Dosya sistemi $B = 4096$ baytlık bloklar kullanıyor. $k = 1000$ numaralı kayıt hangi blokta ve bloğun neresinde başlıyor?

$$\begin{aligned} \text{bayt konumu} &= k \cdot R = 1000 \times 100 = 100\,000, \\ \text{blok} &= \left\lfloor \frac{100\,000}{4096} \right\rfloor = 24, \\ \text{blok içi konum} &= 100\,000 - 24 \times 4096 = 100\,000 - 98\,304 = 1\,696. \end{aligned}$$

Kaydın son baytı $100\,099$'dur ve $\lfloor 100\,099/4096 \rfloor = 24$ de olduğu için kaydın tamamı 24 numaralı göreli blokta, 1696 ile 1795 arasındaki baytlardadır: tek bir blok okuması yeter. C'de bu yalnızca `lseek(fd, 1000 * 100, SEEK_SET)` ve ardından `read(fd, rec, 100)` çağrısıdır; bölmeyi işletim sistemi yapar.

7.3 İndeksli erişim

Büyük dosyalarda doğrudan erişim bile *hangi* kayıt numarasının okunacağını bilmeyi gerektirir. Bunu bir **indeks** (index) çözer: (anahtar, blok numarası) çiftlerinden oluşan küçük, sıralı bir dosya; örneğin öğrenci soyadları ve her öğrencinin kaydını tutan blok. Bir kaydı bulmak için işletim sistemi ya da uygulama indekste arar (küçük ve sıralı olduğu için hızlıdır) ve sonra tek bir bloğu doğrudan okur.

İndeksin kendisi çok büyürse o da indekslenir (çok düzeyli indeks). Bu fikir B-ağaçları gibi veritabanı indekslerinin, IBM'in klasik ISAM'ının (indexed sequential access method, indeksli sıralı erişim yöntemi) ve dosya sistemlerinin bir dosyanın kendi bloklarını bulma yönteminin temelinde yatar.

7.4 Uygulama: C'de sıralı ve doğrudan erişim

Örnek 4, 32 baytlık beş sabit boyutlu kayıttan oluşan bir dosya oluşturur, kayıtları sırayla okur, doğrudan 3. kayda atlar (bayt $3 \times 32 = 96$) ve 1. kaydın üzerine yerinde yazar.

Listede programın özü var; her çağrının dönüş değerini de denetleyen tam sürüm `file_access.c` dosyasıdır.

```c
#define RECSZ 32                    /* kayıt başına bayt */
int fd = open("students.dat", O_CREAT | O_RDWR | O_TRUNC, 0644);
/* ... 5 kayıt yazılır: "0 Ayse", "1 Mehmet", ... "4 Zeynep" */

lseek(fd, 0, SEEK_SET);             /* 1) sıralı        */
while (read(fd, rec, RECSZ) == RECSZ)
    printf("  [%s]", rec);

lseek(fd, 3 * RECSZ, SEEK_SET);     /* 2) doğrudan: bayt 96 */
read(fd, rec, RECSZ);

lseek(fd, 1 * RECSZ, SEEK_SET);     /* 3) 1. kaydın üzerine */
write(fd, rec, RECSZ);
close(fd);
/* programın tamamı: code/file_access.c */
```

```
$ ./file_access
sequential:  [0 Ayse]  [1 Mehmet]  [2 Elif]  [3 Can]  [4 Zeynep]
direct: record 3 at byte 96 -> [3 Can]
after update: record 1 -> [1 Mehmet (updated)]
file size: 160 bytes
```

`read()`'in ilerlettiği ve `lseek()`'in ayarladığı dosya işaretçisi, süreç başına açık dosya tablosunda tutulan "geçerli konumun" ta kendisidir. Dosya $5 \times 32 = 160$ bayt uzunluğundadır; 1. kaydın üzerine yazmak dosyanın boyutunu değiştirmedi ve diğer kayıtlara dokunmadı.

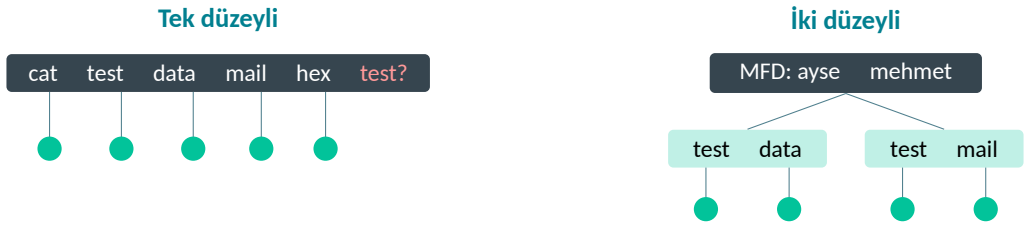
8 Dizin Yapıları

Bir disk milyonlarca dosya tutabilir. **Dizin** (directory, klasör), dosya adlarını onların dosya denetim bloklarına eşleyen bir simge tablosudur. Dizinler üzerindeki işlemler arasında bir dosyayı aramak, dosya oluşturmak ve silmek, bir dizini listelemek, yeniden adlandırmak ve tüm dosya sistemini dolaşmak (örneğin yedekleme için) vardır. Dizinlerin düzenlenme biçimi birkaç aşamada gelişmiştir.

8.1 Tek düzeyli ve iki düzeyli dizinler

Tek düzeyli dizinde (single-level directory) tüm kullanıcıların tüm dosyaları tek bir dizindedir. Basittir, ama her ad benzersiz olmalıdır: iki kullanıcı da `test` adlı bir dosya isterse biri kaybeder. Yüzlerce dosya olunca tek bir liste yönetilemez hâle gelir.

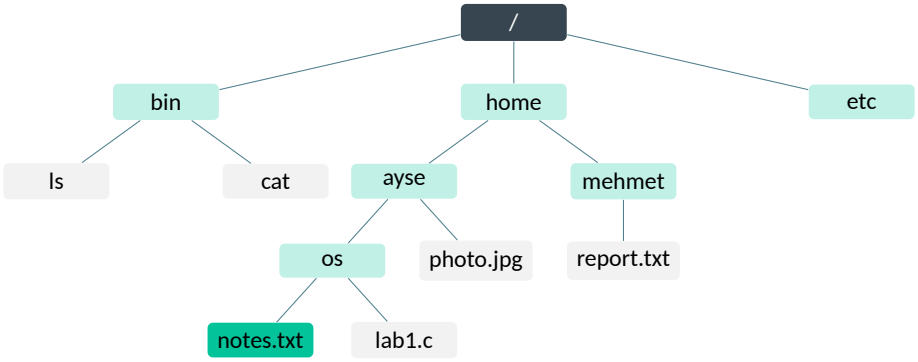
İki düzeyli dizin (two-level directory), her kullanıcıya bir *ana dosya dizininde* (master file directory, MFD) listelenen özel bir *kullanıcı dosya dizini* (user file directory, UFD) verir. Artık iki kullanıcının da `test` adlı bir dosyası olabilir; bir dosya `/ayse/test` gibi bir yolla adlandırılır. Kullanıcılar birbirinden yalıtılmıştır: koruma için iyi, ama dosya paylaşmak istediklerinde zahmetli; ayrıca bir kullanıcının kendi dosyalarını gruplamasının hâlâ bir yolu yoktur.



Şekil 14: Tek düzeyli: herkes için tek bir ad alanı (ikinci `test` çakışıyor). İki düzeyli: her kullanıcıya bir dizin.

8.2 Ağaç yapılı dizinler

Doğal genelleme bir **ağaçtır** (tree): bir dizin, istenen derinliğe kadar dosyalar *ve* başka dizinler (alt dizinler) içerebilir. Her modern işletim sistemi bir ağaç ya da aşağıdaki genellemelerden birini kullanır. Her sürecin bir **geçerli çalışma dizini** (current working directory) vardır.



Şekil 15: Ağaç yapılı bir dizin (soluk: dizinler; gri: dosyalar).

Bir dosya iki biçimde adlandırılabilir:

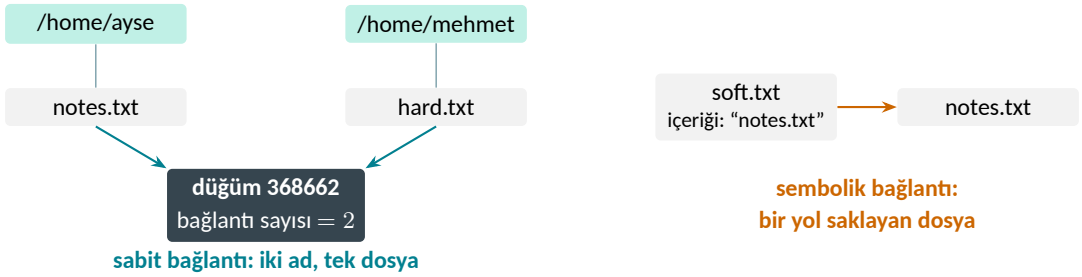
- `/home/ayse/os/notes.txt` bir **mutlak yoldur** (absolute path): kök `/` ile başlar, yol üzerindeki her dizini sayar ve her yerden çalışır.
- **Göreli yol** (relative path) geçerli dizinden başlar. Örneğin geçerli dizin `/home/ayse` olduğunda aynı dosyanın göreli yolu `os/notes.txt` olur.

Her dizinde iki özel girdi bulunur: `.` (dizinin kendisi) ve `..` (üst dizini). `/home/ayse/os` dizininden Mehmet'in raporu `../../mehmet/report.txt` olur. Kabukta `~`, ev dizininizin kısaltmasıdır. Windows aynı fikri sürücü harfleri ve ters bölü işaretleriyle kullanır: `C:\Users\ayse\os\notes.txt`. 9. haftada bu ağaçta `pwd`, `cd` ve `ls` ile dolaşacaksınız.

8.3 Döngüsüz çizge dizinler: sabit ve sembolik bağlantılar

Bir ağaç *paylaşımı* ifade edemez: Ayşe ile Mehmet aynı proje üzerinde çalışıyorsa proje dosyası ikisinin dizininde de görünmelidir; zamanla birbirinden farklılaşan iki kopya olarak değil, tek bir dosya olarak. **Döngüsüz çizge dizin** (acyclic-graph directory), döngü oluşmadığı sürece bir dosyanın ya da alt dizinin birden çok dizinde yer almasına izin verir. UNIX ve Linux iki tür **bağlantı** (link) sunar:

- **Sabit bağlantı** (hard link), *aynı* düğüme işaret eden ikinci bir dizin girdisidir. İki ad eşittir; hiçbiri "asıl" değildir. Düğüm, kendisine işaret eden dizin girdilerinin sayısını, yani **bağlantı sayısını** (link count) tutar. Bir adı silmek (`unlink`) sayıyı bir azaltır; veri ancak sayı 0'a indiğinde ve hiçbir süreç dosyayı açık tutmadığında serbest bırakılır. Sabit bağlantılar dosya sistemi sınırını aşamaz ve (Linux'ta) dizinlere işaret edemez.
- **Sembolik bağlantı** (symbolic link, soft link), içeriği bir *yol adı* olan küçük, ayrı bir dosyadır. Onu açmak, işletim sisteminin bu yolu izlemesine yol açar. Dizinlere ve başka dosya sistemlerine işaret edebilir, ama hedef silinir ya da taşınırsa bağlantı *boşta kalır* (dangling).



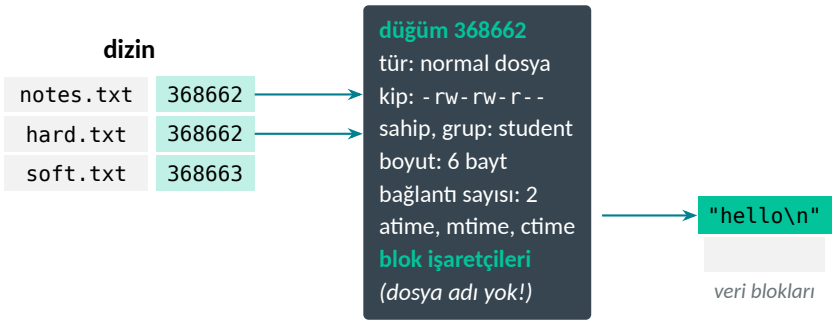
Şekil 16: Sabit bağlantı aynı düğüm için başka bir addır; sembolik bağlantı bir yol içeren dosyadır.

8.4 Genel çizge dizinler

Bağlantıların bir üst dizin de dahil *herhangi bir* dizine işaret etmesine izin verilirse dizin yapısı **döngüler** (cycles) içerebilir. İki sorun ortaya çıkar: dosya sistemini dolaşan bir program (arama, yedekleme) sonsuza dek dönebilir; "silinmiş" bir alt ağaca kendi içinden hâlâ başvurulabilir, bu yüzden bağlantı sayısı hiç 0'a inmez ve pahalı bir çöp toplama yapılmadan yeri asla boşaltılamaz. Uygulamadaki sistemler genel çizgelerden kaçınır: Linux dizinlere sabit bağlantıyı yasaklar ve bir sembolik bağlantı zinciri döngüye girdiğinde yol çözümleme `ELOOP` hatasıyla ("too many levels of symbolic links") durur.

9 Linux Ne Gösterir: Düğümler ve Bağlantılar

Linux'ta bir dizin, (ad, düğüm numarası) çiftlerinden oluşan bir tablodur. Bir dosyayla ilgili diğer her şey (türü, izinleri, sahibi, boyutu, zaman damgaları, bağlantı sayısı ve veri bloklarının işaretçileri) onun **düğümünde** (inode) saklanır. Düğüm dosyanın adını *içermez*; bir düğümün birden çok adı olabilmesinin nedeni budur.



Şekil 17: Dizin girdileri düğümlere, düğümler veri bloklarına işaret eder.

Yararlı komutlar: `ls -i` düğüm numaralarını yazdırır, `ls -l` çıktısının ikinci sütunu bağlantı sayısıdır, `stat` düğümün tamamını gösterir ve `df -i` kaç düğümün boş olduğunu gösterir. Bilmeye değer bir sonuç: aynı dosya sistemi içinde `mv` yalnızca dizin girdilerini değiştirir; bu yüzden dev bir dosyayı taşımak bile anında biter, düğüm ve veri yerinde kalır.

9.1 Uygulama: `ls -li`, `ln`, `ln -s` ve `stat`

`code/links_demo.sh` betiği aşağıdaki komutları çalıştırır (daha iyisi: boş bir dizinde komutları tek tek yazın). Test makinemizde sıradan bir kullanıcı olarak yakalanan çıktı:

```
$ echo "hello" > notes.txt
$ ls -li notes.txt
368662 -rw-rw-r-- 1 student student 6 Oct  8 12:20 notes.txt
$ ln notes.txt hard.txt
$ ln -s notes.txt soft.txt
$ ls -li
total 8
368662 -rw-rw-r-- 2 student student 6 Oct  8 12:20 hard.txt
368662 -rw-rw-r-- 2 student student 6 Oct  8 12:20 notes.txt
368663 lrwxrwxrwx 1 student student 9 Oct  8 12:20 soft.txt -> notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6           Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 368662      Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1001/ student)   Gid: ( 1002/ student)
Access: 2026-10-08 12:20:48.530146698 +0000
Modify: 2026-10-08 12:20:48.530146698 +0000
Change: 2026-10-08 12:20:48.534146698 +0000
 Birth: 2026-10-08 12:20:48.530146698 +0000
$ rm notes.txt
$ cat hard.txt
hello
$ cat soft.txt
cat: soft.txt: No such file or directory
$ ls -li
total 4
368662 -rw-rw-r-- 1 student student 6 Oct  8 12:20 hard.txt
368663 lrwxrwxrwx 1 student student 9 Oct  8 12:20 soft.txt -> notes.txt
```

Neye dikkat edelim:

- `notes.txt` ve `hard.txt` *aynı* düğüm numarasına (368662) ve 2 bağlantı sayısına sahip: iki ad, tek dosya.
- `soft.txt`'nin kendi düğümü (368663), türü `l` ve boyutu 9'dur: tam olarak sakladığı `notes.txt` dizgesinin uzunluğu.
- `stat` çıktısında *Change* (ctime, *düğümün* son değişikliği), *Modify*'dan (mtime, *verinin* son değişikliği) birkaç milisaniye sonradır: sabit bağlantıyı oluşturmak içeriği değil, düğümdeki bağlantı sayısını güncelledi.
- `notes.txt` adı `rm` ile silindikten sonra veriye `hard.txt` üzerinden hâlâ erişilebilir (bağlantı sayısı yine 1); `soft.txt` ise boşta kalmıştır: sakladığı yol artık yoktur.

Bu oturum, 10. haftanın (dosya işlemleri: `cp`, `mv`, `rm`, `ln`) ve düğümde saklanan `-rw-rw-r--` izin dizgesinin ayrıntılı olarak açıklanacağı 11. haftanın bir ön izlemesidir.

10 Özet

Sekiz cümlede 7. hafta

1. Sanal bellek her sürece büyük bir adres alanı verir; bu alanın yalnızca kullanılan sayfaları RAM'dedir. Bunu mümkün kılan yerelliktir.
2. İsteğe bağlı sayfalama bir sayfayı ilk kullanımında yükler; geçerli–geçersiz biti MMU'ya bir sayfanın bellekte olup olmadığını söyler.
3. Geçersiz bir sayfaya erişim işletim sistemine tuzak üretir (sayfa hatası); işletim sistemi sayfayı yükler ve komutu yeniden başlatır; EAT $= (1-p)t_m + p\,t_{\text{fault}}$ olduğundan p çok küçük olmalıdır.
4. Yazınca kopyala, `fork()` sonrasında sayfaları süreçlerden biri yazana kadar paylaştırır.
5. Boş çerçeve yoksa bir değiştirme algoritması kurban seçer: FIFO (örneğimizde 15 hata, Belady anomalisi olabilir), OPT (9, en iyi ama geleceği ister), LRU (12, saat algoritmasıyla yaklaşılır).
6. Boşa çalışma, süreçlerin çerçeve sayısı çalışma kümelerinden az olduğunda ortaya çıkar; çare bellekte daha az süreç tutmaktır.
7. Dosya, öznitelikleri olan adlandırılmış bir veri topluluğudur; bir kez açılır, sonra sıralı olarak, doğrudan (blok hesabıyla) ya da bir indeks aracılığıyla erişilir.
8. Dizinler tek düzeyliden iki düzeyliye, ağaçlara ve döngüsüz çizgelere evrildi; Linux'ta dizin girdileri adları düğümlere eşler ve sabit ve sembolik bağlantılar bir dosyaya birden çok ad verir.

11 Sözlük

Belady anomalisi (Belady's anomaly) Daha çok çerçevenin daha çok sayfa hatasına yol açması; FIFO'da görülebilir.

Boşa çalışma (thrashing) Süreçlerin yürütmekten çok sayfalamaya zaman harcadığı durum.

Çalışma kümesi (working set) Son Δ başvuruda kullanılan farklı sayfaların kümesi.

Değişiklik (kirli) biti (dirty / modify bit) Bir sayfaya yazıldığında kurulan donanım biti; temiz sayfaların diske geri yazılması gerekmez.

Dosya denetim bloğu / düğüm (file control block / inode) Bir dosyanın özniteliklerini ve blok konumlarını tutan, dosya başına yapı.

Geçerli–geçersiz biti (valid–invalid bit) Bir sayfanın bellekte olup olmadığını gösteren sayfa tablosu biti.

İkinci şans (saat) algoritması (second-chance / clock) Başvuru bitini kullanarak çerçeveleri bir çember üzerinde tarayan LRU yaklaşımı.

Sabit bağlantı (hard link) Var olan bir düğüm için ek bir dizin girdisi.

Sanal bellek (virtual memory) Programın adres alanının fiziksel bellekten ayrılması; böylece adres alanı RAM'den büyük olabilir.

Sayfa hatası (page fault) Bir süreç bellekte olmayan bir sayfaya eriştiğinde üretilen tuzak.

Sembolik bağlantı (symbolic link) İçeriği başka bir dosyanın yolu olan dosya.

Yedek depo (backing store) RAM'de olmayan süreç sayfalarını tutan disk alanı (takas alanı).

12 Kendiniz Deneyin

Kendi makinenizde sayfa hataları, sayfa değiştirme ve bağlantılar

Linux (laboratuvar bilgisayarları, WSL ya da bir sanal makine), `code/` klasöründe:

```
$ make                          # üç C programını derler
$ ./page_faults                 # 64'ü 256 MB yapın: ne olur?
$ /usr/bin/time -v ./cow_fork   # "Minor ... page faults" satırına bakın
$ python3 page_replacement.py   # refs'i değiştirip yeniden çalıştırın
$ mkdir lab && cd lab && bash ../links_demo.sh
```

1. `page_faults.c` içindeki ilk döngüde yalnızca her *ikinci* sayfaya dokunun. Kaç hata beklersiniz, kaç hata alırsınız?
2. `page_replacement.py` içindeki başvuru dizisini kendi dizinizle değiştirin ve elle çizdiğiniz FIFO, OPT ve LRU tablolarını doğrulayın.
3. Bir dosyaya sabit bağlantı yapın, dosyayı bir adıyla düzenleyip öbür adıyla açıp bakın. Sonra `ln` komutunu bir dizin üzerinde deneyin: hangi hatayı alıyorsunuz?
4. `ls -li /` ve `stat /` komutlarını çalıştırın. Sisteminizde kök dizinin düğüm numarası kaçtır?

13 Tekrar Soruları

Kavramsal

1. Sanal bellek nedir ve bir program, sayfalarının yalnızca bir kısmı RAM'deyken neden çalışabilir?
2. Hevesli yükleme ile isteğe bağlı sayfalama arasındaki farkı açıklayın. İsteğe bağlı sayfalamanın bedeli nedir?
3. Geçerli–geçersiz biti ne anlama gelir? Bir sayfanın i olarak işaretlenmesi için iki farklı neden verin.
4. Sayfa hatası işlemenin altı adımını sayın. Hataya yol açan komut neden yeniden başlatılmalıdır?
5. Yazınca kopyala yöntemini açıklayın. `fork()` ardından `exec()` çağrısını neden ucuzlatır?
6. OPT neden gerçeklenemez? Yine de neden yararlıdır?
7. Belady anomalisi nedir? FIFO, OPT ve LRU'dan hangileri bundan etkilenebilir?
8. Boşa çalışmayı ve ona yol açabilecek kısır döngüyü anlatın. Çalışma kümesi modeli nasıl yardımcı olur?
9. Sıralı, doğrudan ve indeksli erişimi karşılaştırın; her biri için bir uygulama örneği verin.
10. Sabit ve sembolik bağlantıları karşılaştırın: özgün ad silindiğinde her birine ne olur?

Sayısal

11. $t_m = 200$ ns ve $t_{\text{fault}} = 8$ ms iken $p = 0.0001$ için EAT'yi hesaplayın.
12. $t_m = 100$ ns ve $t_{\text{fault}} = 5$ ms iken yavaşlamayı %20'nin altında tutan en büyük p nedir?
13. $1, 2, 3, 1, 4, 1, 2$ başvuru dizisi ve 3 çerçeve için FIFO, LRU ve OPT'nin sayfa hatalarını sayın.
14. $7, 0, 1, 2, 0, 3, 0, 4$ başvuru dizisi ve 4 çerçeve için FIFO ve LRU'nun hatalarını sayın.
15. Kayıtlar 64 bayt, bloklar 4096 bayttır. 200 numaralı kayıt hangi göreli blokta ve hangi konumda başlar?
16. Bir dosyada 250 baytlık kayıtlar ve 1024 baytlık bloklar var. 40 numaralı kayıt tek bir bloğa sığar mı?

Sayısal soruların yanıtları

11. EAT $= 0.9999 \times 200 + 0.0001 \times 8\,000\,000 = 199.98 + 800 = 999.98$ ns $\approx 1\ \mu$s; RAM'den beş kat yavaş.
12. $100 + p\,(5\,000\,000 - 100) < 120$ olmalı, yani $p < 20/4\,999\,900 \approx 4.0 \times 10^{-6}$: en fazla 250.000 erişimde bir hata.
13. FIFO: $1, 2, 3$ (3 hata), 1 isabet, 4 1'in yerine, 1 2'nin yerine, 2 3'ün yerine geçer: 6 hata. LRU: 4 2'nin (en uzun süredir kullanılmayan) yerine geçer, 1 isabet, 2 3'ün yerine geçer: 5 hata. OPT: 4 3'ün (bir daha hiç kullanılmayan) yerine geçer: 4 hata.
14. 4 çerçeveyle $7, 0, 1, 2$ 4 zorunlu hataya yol açar, 0 bir isabettir, 3 ve 4 hatadır. FIFO: 3 7'nin yerine, 0 isabet, 4 0'ın yerine geçer: 6 hata. LRU: 3 7'nin yerine, 0 isabet, 4 1'in yerine geçer: 6 hata.
15. Bayt konumu $200 \times 64 = 12\,800$; blok $\lfloor 12\,800/4096 \rfloor = 3$; blok içi konum $12\,800 - 3 \times 4096 = 512$.
16. 40 numaralı kayıt $40 \times 250 = 10\,000$ ile $10\,249$ arasındaki baytları kaplar. $\lfloor 10\,000/1024 \rfloor = 9$, ama $\lfloor 10\,249/1024 \rfloor = 10$: kayıt bir blok sınırını aşar, bu yüzden onu okumak iki blok gerektirir.

14 Vize Tekrarı: 1--7. Haftalar

8. haftadaki vize sınavı 1. haftadan bu haftaya kadar her şeyi kapsar. Bu listeyi bir kontrol listesi olarak kullanın; her madde, belirtilen haftanın slaytlarında ve ders notlarında yer aldı.

1. hafta: Tarihçe İşletim sistemi ne yapar; toplu işleme sistemleri, çoklu programlama, zaman paylaşımı; boşta duran işlemci tasarımı nasıl yönlendirdi; UNIX (1969) ve Linux (1991).

2. hafta: Temeller Tanımlar ve çekirdek; genişletilmiş makine ve kaynak yöneticisi bakışları; hedefler; kesmeler; bellek hiyerarşisi; ikili mod; sistem çağrıları (`strace`); basit, monolitik, katmanlı, mikroçekirdek, modüler ve hibrit yapılar.

3. hafta: Süreçler İşletim sistemi hizmetleri; sistem çağrısı türleri; süreç ve bellek düzeni; süreç durumları; PCB; bağlam değişimi; `fork()`.

4. hafta: İş parçacıkları, işlemci İş parçacıkları ve süreçler; kullanıcı ve çekirdek iş parçacıkları; çoklu iş parçacığı modelleri; işlemci–G/Ç patlamaları; zamanlama ölçütleri (kullanım, üretilen iş, tamamlanma, bekleme ve yanıt süresi).

5. hafta: Zamanlama, eşzamanlama Gantt şemaları ve ortalama bekleme süresiyle FCFS, SJF/SRTF, Öncelik, Round Robin; yarış durumları; kritik bölge problemi; mutex kilitleri; semaforlar.

6. hafta: Bellek Adres bağlama; mantıksal ve fiziksel adresler; MMU; takas; bitişik yer ayırma; iç ve dış parçalanma; sayfalama ve adres çevirisi; TLB ve EAT; bölümleme.

7. hafta: Sanal bellek, dosyalar İsteğe bağlı sayfalama; sayfa hataları; sayfa hatalarıyla EAT; yazınca kopyala; FIFO, OPT, LRU tabloları; Belady anomalisi; saat algoritması; boşa çalışma ve çalışma kümesi; dosya öznitelikleri ve işlemleri; blok hesabıyla erişim yöntemleri; dizin yapıları; sabit ve sembolik bağlantılar.

Alıştırma yapılacak sayısal soru türleri: Gantt şemaları ve bekleme süreleri (5. hafta); adres çevirisi ve TLB ile EAT (6. hafta); sayfa hatalarıyla EAT ve sayfa değiştirme çerçeve tabloları (7. hafta); doğrudan erişim blok hesabı (7. hafta). En iyi çalışma malzemesi, her haftanın notlarının sonundaki çözümlü yanıtlı tekrar sorularıdır.

15 İleriye Bakış: 8. Hafta ve Sonrası

- **8. hafta: Vize sınavı**, 1–7. haftalar. Öğrenci kimliğinizi ve kaleminizi getirin; kesin saat, salon ve kurallar (örneğin hesap makinesiyle ilgili) dersin Telegram kanalında duyurulacak.
- **9. hafta** dersin uygulamalı yarısını başlatır: Linux kabuğu ve bu hafta çizdiğimiz dizin ağacında gezinme (`pwd`, `cd`, `ls`, mutlak ve göreli yollar).
- **10. hafta:** dosya işlemleri (`cp`, `mv`, `rm`, `ln`, `touch`); **11. hafta:** izinler (düğümdeki `-rw-rw-r--` dizgesi).

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley: Bölüm 10 (Virtual Memory), Bölüm 13 (File-System Interface) ve Kısım 14.1 (File-System Structure).
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Bölüm 21, “Beyond Physical Memory: Mechanisms”: `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-beyondphys.pdf`
  - Bölüm 22, “Beyond Physical Memory: Policies”: `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-beyondphys-policy.pdf`
  - Bölüm 39, “Interlude: Files and Directories”: `https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf`
- L. A. Bélády, R. A. Nelson ve G. S. Shedler, “An anomaly in space-time characteristics of certain programs running in a paging machine”, *Communications of the ACM* 12(6), 1969, s. 349–353.